

Capstone Proposal

Compressing LLMs with Holograms

Karapet Khachatryan

karapet.khachatryan@edu.aua.am

Supervisor: Suren Khachatryan

American University of Armenia, Yerevan, Armenia
Bachelors of Computer Science

1 Abstract

This capstone project explores whether the holographical principle can be applied as an experimental compression for large language models, more particularly, for their weight matrices. In classical holography, a three-dimensional object is saved as an interference pattern that contains information about the amplitude and phase. Inspired by holography, this project treats neural network parameters as two-dimensional images, which are composed into a three-dimensional geometry. Next, the proposed geometry is encoded into the holographic diffraction image and then decoded back into the reconstructed matrices. The proposed pipeline includes matrix normalization, matrix-to-surface mapping, VTP point-cloud generation, holographic encoding, amplitude and phase separate storage, reconstruction quality measurement metrics such as RMSE, MAE, etc. The main research question is whether the holographic encoding can be used as a compression tool.

2 Introduction

Large language models have become central to modern artificial intelligence, but their practical use is limited by the size of their parameter sets, memory requirements, and computational cost. Transformer-based models contain many large weight matrices, often repeated across attention and feed-forward layers. These matrices are usually stored directly as numerical arrays, which makes deployment expensive when the model has billions of parameters. For this reason, model compression has become an alternative research direction. Existing compression methods usually aim to reduce the number of stored bits, remove less important weights, approximate matrices with smaller structures, or transfer knowledge from a larger model to a smaller one.

This capstone approaches the problem from a different perspective. Instead of asking only how many parameters can be removed or how many bits each weight can use, it asks whether neural weights can be transformed into another representational domain. The main inspiration is holography. Dennis Gabor introduced holography in 1948 as a method for recording and reconstructing wavefront information rather than simply capturing an ordinary intensity image. In holography, the recorded pattern may look visually meaningless, but it reconstructs the appearance of a theoretical virtual object with high precision. This is important for the project because it suggests a broader idea: information does not always need to be stored in the same form in which it is later used. A complex object can be encoded into an indirect representation and reconstructed afterward.

The project applies the holographic principle to model weights. A weight matrix can be interpreted as a surface, where the row and column indices define spatial coordinates and the weight value defines height, amplitude, depth, or phase. Multiple matrices can also be packed together into a larger spatial representation, or transformed onto curved surfaces such as hemispheres. After this conversion, the resulting object can be encoded into a hologram-like two-dimensional image. Decoding then attempts to reconstruct the original matrix or point cloud from the holographic representation.

The research is not claiming that holographic encoding is already better than standard compression. Rather, it studies whether such an approach can preserve meaningful numerical structure while reducing storage size. Early experiments show that reconstruction can be very sensitive to hologram resolution, depth handling, angle selection, normalization, and whether only amplitude or both amplitude and phase are stored. A zero matrix reconstructs easily because it contains almost no structural variation, while more complex matrices expose the weakness of the current encoding and decoding process. This makes the project valuable as a research exploration: even failed or low-similarity reconstructions help identify what kind of information the holographic pipeline loses.

The main goal of the capstone is therefore to design, implement, and evaluate a hologram-inspired encoding-decoding pipeline for neural weight matrices. The project uses simple geometric objects such as cubes and bell shapes as controlled test cases, then applies similar methods to language model weight matrices. The success of the method is evaluated through reconstruction metrics, visual inspection, file-size comparison, and analysis of whether depth, phase, and matrix structure are preserved.

3 Literature Review

Large Language Models are built mainly from large numerical weight matrices, especially inside Transformer architectures. The Transformer introduced self-attention as a replacement for recurrent and convolutional sequence models, making modern language models highly parallelizable but also heavily dependent on large-scale matrix operations. BERT later showed how stacked Transformer layers could produce powerful language representations, but also made clear that high performance depends on storing and processing many large parameter matrices. This creates the central motivation for model compression: reducing memory, storage, and inference cost while preserving as much model behavior as possible.

Most existing LLM compression research follows four major directions: quantization, pruning, knowledge distillation, and low-rank decomposition. A recent survey by Zhu et al. identifies these as the main families of LLM compression methods and emphasizes that compression is necessary because LLMs are difficult to deploy on resource-limited hardware. Quantization reduces the number of bits used to represent weights or activations. For example, GPTQ introduced a post-training quantization method that uses approximate second-order information and can reduce GPT-style model weights to 3 or 4 bits with limited accuracy loss. SmoothQuant focuses on activation outliers and enables 8-bit weight-and-activation quantization by shifting quantization difficulty from activations to weights. AWQ similarly argues that not all weights are equally important and protects a small percentage of salient weights based on activation statistics.

Pruning removes less important parameters or structures from the model. SparseGPT, for instance, demonstrated that very large GPT-family models could be pruned to around 50

However, these methods compress LLMs mostly within the original neural-network framework. Quantization keeps the matrix layout but lowers precision; pruning deletes weights; distillation transfers behavior into a smaller model; and low-rank factorization approximates matrices with simpler components. The proposed capstone project explores a different question: whether LLM parameters can be treated as a spatial signal and encoded through a holography-inspired representation. In this view, each weight matrix is not only a table of numbers but also a two-dimensional surface or layer. A stack of such layers can be interpreted as a three-dimensional object, which can then be encoded into a two-dimensional hologram-like pattern and decoded back into an approximate reconstruction.

The theoretical inspiration comes from holography. Dennis Gabor introduced holography in 1948 as a method for recording and reconstructing wavefront information. Unlike ordinary images, which store only intensity from one viewpoint, holograms encode information about a wavefield through interference. This is important for the capstone because the project is not simply image compression; it asks whether a lower-dimensional encoded representation can preserve enough information to reconstruct a higher-dimensional structure. In physical holography, a two-dimensional recording can reproduce the appearance of a three-dimensional object. In the capstone analogy, a two-dimensional hologram-like file is used to reconstruct a three-dimensional representation of model weights.

Digital and computer-generated holography provide the closest technical foundation. Digital holograms are often complex-valued, containing amplitude and phase information, and their signal properties differ from natural images. This makes ordinary image compression difficult, because phase and high-frequency interference patterns may be essential for accurate reconstruction. JPEG Pleno Holography was introduced as the first international standard focused on efficient compression and representation of holographic content, showing that hologram compression is now recognized as a separate problem from ordinary image compression. Research on hologram compression also notes that raw holographic data can be extremely large, especially for holographic video, motivating specialized compression methods.

Recent work has also applied neural networks to hologram compression and reconstruction. Shi et al. proposed neural compression for complex-valued hologram images and videos, showing that learned compression can be applied directly to holographic data rather than treating it as an ordinary image. Other work on deep-learning-based computer-generated holography reviews data-driven, physics-driven, and jointly optimized methods, suggesting that neural networks can assist both hologram generation and reconstruction. This is relevant for the capstone because the current project may begin with algorithmic encoding and decoding, but future improvement could involve a learned decoder trained to reconstruct matrices or point clouds more accurately from hologram-like encodings.

Compressive holography is especially relevant because it studies reconstruction of three-dimensional information

from fewer measurements. Brady et al. describe compressive holography as a method where compressed sensing enables reconstruction using fewer measurements than the number of reconstructed signal values. Later work applies compressive holography to three-dimensional reconstruction from two-dimensional holographic data, often relying on sparsity assumptions. This connects strongly to the capstone’s core hypothesis: if LLM weight structures contain redundancy, then a compressed holographic representation might preserve the most important information even when the encoded file is smaller than the original matrix or VTP representation.

At the same time, the literature also shows why the problem is difficult. True three-dimensional reconstruction from holographic data is not simply an exact inverse operation. Birdi et al. explain that replay or back-propagation in digital holography is closer to a Hermitian transpose than a true inverse, meaning that naive reconstruction only approximates the original 3D object. This directly explains a major challenge in the capstone experiments: a decoded point cloud or matrix may visually resemble the original object while still having high numerical error. Therefore, evaluation must use strict metrics such as RMSE, MAE, cosine similarity, nearest-neighbor distances, voxel IoU, and downstream model-performance tests when possible.

Overall, the literature suggests that the capstone project is exploratory but well motivated. LLM compression research proves that model weights contain redundancy and can often be stored more efficiently. Holography and compressive imaging show that higher-dimensional spatial information can be encoded into lower-dimensional wave-like representations. Neural hologram compression further shows that learned methods can improve reconstruction quality when classical decoding is insufficient. The research gap is that these ideas are rarely applied directly to LLM weight matrices. Therefore, this capstone contributes by testing whether holography-inspired encoding can serve as an alternative compression framework for neural network parameters, beginning with simple 3D objects and moving toward real LLM weight matrices.

4 Problem Formulation

Large Language Models require a large amount of storage because their knowledge is represented through millions or billions of numerical parameters. These parameters are mainly stored as weight matrices inside Transformer layers. As models become larger, the cost of saving, transferring, and deploying them also increases. Existing compression methods such as quantization, pruning, knowledge distillation, and low-rank decomposition reduce model size, but they usually remain within the traditional structure of neural networks. This project formulates a different research problem: whether the parameters of a language model can be represented as a spatial object and compressed through a holography-inspired encoding process.

The central idea of this capstone is based on the analogy between holography and neural network weight storage. In physical holography, information about a three-dimensional object can be encoded into a two-dimensional hologram pattern. Although the hologram is stored on a two-dimensional surface, it preserves enough information to reconstruct the visual structure of the original object. Similarly, an LLM can be viewed as a layered structure: each weight matrix is a two-dimensional surface, and a sequence of such matrices can be interpreted as a three-dimensional object. The problem is whether this three-dimensional representation of model parameters can be encoded into a smaller two-dimensional hologram-like representation and later reconstructed with acceptable accuracy.

Formally, let a model contain a set of weight matrices:

$$W = \{W_1, W_2, \dots, W_n\},$$

where each W_i represents a matrix from one part of the model, such as an attention layer or feed-forward layer. These matrices can be transformed into a spatial representation P , such as a point cloud or surface, where matrix indices define spatial coordinates and matrix values define height, depth, intensity, or density. The goal is to construct an encoding function:

$$E(P) = H,$$

where H is a compact hologram-like representation, usually stored as an image or a small set of images with possible metadata. Then, a decoding function:

$$D(H) = \hat{P}$$

attempts to reconstruct the original spatial object. Finally, the reconstructed object $\hat{P}$ can be converted back into approximate weight matrices:

$$\hat{W} = \{\hat{W}_1, \hat{W}_2, \dots, \hat{W}_n\}.$$

The main research problem is to determine whether $\hat{W}$ is close enough to W while the encoded representation H is significantly smaller than the original stored model parameters.

Therefore, the project studies a compression–reconstruction trade-off. A successful method should reduce storage size while maintaining reconstruction quality. However, this is challenging because holographic encoding may lose numerical precision, depth information, or fine structural details. For ordinary images or three-dimensional objects, a visually similar reconstruction may be acceptable. For neural network weights, small numerical errors may still affect model behavior. Because of this, the evaluation cannot rely only on visual similarity. It must also include numerical metrics such as Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), cosine similarity, nearest-neighbor distance, voxel Intersection over Union (IoU), and compression ratio. In future extensions, the reconstructed model should also be tested by comparing its actual inference performance with the original model.

This project begins with simple geometric objects, such as cubes and bell-shaped surfaces, because they allow easier visual and numerical evaluation of the holographic encoding and decoding pipeline. These experiments help test whether the method can preserve basic spatial structure under compression. After that, the same framework can be applied to matrices that resemble or come from real LLM layers. This step-by-step structure is important because directly applying the method to full LLMs would make it difficult to identify whether errors come from the holographic algorithm, the spatial conversion, the decoder, or the model itself.

The problem can be summarized through the following research question:

Can a holography-inspired two-dimensional encoding preserve enough information from three-dimensional representations of LLM weight matrices to provide meaningful compression with acceptable reconstruction accuracy?

From this main question, the project has several subproblems:

1. **Representation problem:** How should weight matrices be converted into a spatial object suitable for holographic encoding?
2. **Encoding problem:** What hologram-like encoding method can store the spatial representation compactly?
3. **Decoding problem:** How accurately can the original object or matrix be reconstructed from the compressed hologram?
4. **Evaluation problem:** Which metrics best measure whether the reconstruction is good enough for model-compression purposes?
5. **Compression trade-off problem:** How much file-size reduction can be achieved before reconstruction quality becomes unacceptable?

The expected outcome is not necessarily to prove that holography is already a practical replacement for existing LLM compression methods. Instead, the goal is to investigate whether this alternative representation has scientific potential. If the reconstructed matrices preserve structure well under compression, the project may suggest a new direction for neural network storage. If reconstruction quality remains weak, the project will still identify the limitations of applying holographic principles to LLM compression and clarify what improvements may be needed, such as phase-aware encoding, multi-view holograms, better depth recovery, or AI-assisted decoders.

5 Proposed Methodology

The proposed methodology follows a step-by-step experimental structure. Instead of immediately applying the holographic compression idea to Large Language Model weight matrices, the project first focuses on reconstructing simple three-dimensional objects. This decision is important because simple objects, such as cubes and bell-shaped surfaces, provide a controlled environment where reconstruction quality can be visually and numerically evaluated. If the method cannot preserve the geometry of basic three-dimensional objects, then applying it directly to complex neural network parameters would not be meaningful.

The first stage of the methodology is therefore the validation of a holography-inspired encoding and decoding pipeline on VTP point-cloud objects. A VTP file represents a three-dimensional object as a set of points in space. These points are read from the input file and normalized so that objects of different sizes can be processed within a common coordinate system. This normalization step allows the algorithm to apply the same holographic parameters, such as hologram size, depth range, and spatial extent, across different objects.

After normalization, the object is observed from one or more virtual viewpoints. The implementation supports a single view, six cube-like views, and circular multi-view configurations. In the main experiments, the six-view cube configuration is especially useful because it captures the object from the front, back, left, right, top, and bottom directions. This multi-view strategy is used because a single two-dimensional projection may lose important depth information. By encoding the object from several directions, the system increases the chance that the true three-dimensional structure can be recovered during decoding.

For each view, the normalized point cloud is rotated into the corresponding view coordinate system. The depth dimension is then divided into several depth bins. Points belonging to each depth slice are projected onto a two-dimensional complex plane. To imitate holographic behavior, the projected points are assigned complex values with phase information, and each depth slice is propagated using an angular spectrum propagation method. The propagated slices are summed together to form a complex hologram field for that view. Thus, the encoded representation is not a normal image of the object, but a wave-based representation that combines spatial position, depth, amplitude, and phase.

The encoded output for each view is stored as two image files: one for amplitude and one for phase. The amplitude is saved using a logarithmic scale, while the phase is mapped into a normalized image range. Alongside these image files, the pipeline stores a metadata file containing the original point count, object bounds, normalization parameters, hologram size, depth-bin count, wavelength, pixel pitch, view rotations, and other reconstruction parameters. Therefore, the compressed package consists of amplitude images, phase images, and metadata rather than a direct copy of the original VTP point cloud.

The second stage is decoding. The decoder reconstructs the complex hologram field from the stored amplitude and phase images. For each view and each depth bin, the field is back-propagated to estimate where strong object evidence may exist at that depth. Instead of directly converting the brightest pixels into reconstructed points, the method uses a confidence-volume approach. In this approach, candidate points from each back-propagated view are mapped into a shared three-dimensional voxel grid. Each view contributes votes to this grid, producing a confidence volume that represents how likely each spatial location is to belong to the original object.

This confidence-volume decoder is a key part of the methodology. Older decoding strategies may produce noisy points because they select bright peaks independently from each view. In contrast, the confidence-volume method requires evidence from multiple views before accepting a voxel as reliable. The decoder applies a minimum view-support condition, removes low-confidence voxels, and can also apply connected-component filtering to remove isolated noise. As a result, the final reconstructed object is produced only from the voxels that survive these confidence and consistency checks.

The reconstruction is then converted back into a VTP point cloud. Importantly, the method is quality-first rather than count-first. It does not automatically force the reconstructed point cloud to have the same number of points as the original object. If the decoder finds fewer reliable voxels, it outputs fewer points instead of inventing artificial points. This is useful for strict reconstruction testing because forcing the original point count could make the object look denser while also adding geometrically incorrect points.

The third stage is evaluation. Since the first experiments use simple three-dimensional objects, reconstruction quality can be evaluated both visually and numerically. The pipeline computes strict geometric metrics between the original and reconstructed VTP files. These include nearest-neighbor distance from original to reconstruction, nearest-neighbor distance from reconstruction to original, RMSE, percentile errors, symmetric Hausdorff distance, precision, recall, F1 score under several distance thresholds, voxel Intersection over Union, and compression ratio. For cube-like objects, additional surface metrics can also be used, such as the percentage of reconstructed points outside the original bounding box and the distance to the nearest cube face.

This methodology supports the broader goal of the capstone project. Once the pipeline demonstrates that simple three-dimensional objects can be encoded, compressed, and reconstructed with measurable accuracy, the same idea can be extended to neural network weight matrices. In that later stage, a weight matrix can be interpreted as a two-dimensional surface where row and column indices define spatial coordinates and the weight value defines height, depth, or intensity. Multiple matrices from a model can then be stacked or packed into a three-dimensional representation. The holography-inspired encoder can compress this representation into amplitude and phase images, and the decoder can attempt to reconstruct the approximate original matrices.

Therefore, the proposed methodology is intentionally progressive. It begins with controlled three-dimensional shapes, where the success or failure of reconstruction is easier to interpret. Then it moves toward model-weight representations, where the same compression-reconstruction principle can be tested on data closer to the final capstone objective. This staged approach makes the research more reliable because it separates the basic geometric reconstruction problem from the more complex question of whether reconstructed neural network weights preserve model behavior.

6 Framework Design and Parameter Exploration

After defining the general methodology, this project implements an experimental framework for testing holography-inspired compression and reconstruction. The purpose of the framework is not only to run one fixed experiment, but also to explore how different encoding and decoding settings affect compression ratio, reconstruction accuracy, and geometric stability. This is important because the proposed approach is experimental: there is no single known optimal configuration for converting a three-dimensional object, or later a set of neural network weight matrices, into a hologram-like representation.

The framework is organized as a full pipeline that takes an input VTP point cloud, encodes it into a hologram package, decodes the package back into a reconstructed VTP point cloud, and then generates strict evaluation reports. Therefore, the implementation supports three main research activities: encoding, decoding, and quantitative comparison. In addition, an automated **run-all** mode connects all stages into one experiment, making it possible to repeatedly test different parameter configurations.

6.1 Framework Overview

The implemented framework follows the structure:

Input VTP object $\rightarrow$ Holographic encoding $\rightarrow$ Amplitude/phase image package $\rightarrow$
 $\rightarrow$ Confidence-volume decoding $\rightarrow$ Reconstructed VTP object $\rightarrow$ Strict metric report

The input object is represented as a VTP point cloud. This format is useful for the first stage of the project because it allows simple three-dimensional objects, such as cubes or bell-shaped surfaces, to be represented as sets of spatial points. The framework first reads the point cloud, normalizes its coordinates, and prepares it for holographic encoding. Normalization is necessary because different objects may have different physical sizes or coordinate ranges, while the holographic encoder expects the object to fit within a controlled spatial extent.

The encoded representation is stored as a collection of image files and metadata. For each view, the framework saves one amplitude image and one phase image. The metadata file stores the information needed for reconstruction, including the original point count, original bounds, normalization center and scale, hologram size, number of depth bins, optical parameters, view mode, and view rotation matrices. This design reflects the idea that a hologram should not only store intensity-like information, but also phase information, since phase plays an important role in wave-based reconstruction.

6.2 Encoding Module

The encoding module is responsible for transforming a normalized three-dimensional point cloud into a hologram-like representation. First, the object is observed from a selected virtual view. The framework supports multiple view configurations, such as a single front view, a six-direction cube view, and circular multi-view settings. The six-view configuration is especially useful for three-dimensional reconstruction because it captures information from the front, back, left, right, top, and bottom directions.

For each view, the object points are rotated into the view coordinate system. The depth axis is then divided into a fixed number of depth bins. Each depth bin represents a slice of the object at a certain distance from the hologram plane. Points in each depth slice are projected onto a two-dimensional complex plane. Random phase values are assigned to the projected points, and the slice is propagated using an angular spectrum propagation method. The propagated fields from all depth slices are then summed to form the final complex hologram field for that view.

The encoding process can be summarized as:

$$P \rightarrow P_v \rightarrow \{P_{v,1}, P_{v,2}, \dots, P_{v,d}\} \rightarrow \{F_{v,1}, F_{v,2}, \dots, F_{v,d}\} \rightarrow H_v,$$

where P is the normalized point cloud, P_v is the point cloud rotated into a specific view, $P_{v,i}$ is the set of points belonging to depth slice i , $F_{v,i}$ is the propagated complex field for that slice, and H_v is the final hologram field for view v .

The framework provides several encoding parameters:

- **holo-size**: controls the resolution of the generated hologram images.
- **depth-bins**: controls how many depth slices are used to represent the object.
- **views**: controls the virtual camera configuration, such as **single**, **cube6**, or circular views.

- **wavelength**: controls the simulated wavelength used in wave propagation.
- **pixel-pitch**: controls the physical spacing between hologram pixels.
- **extent**: controls the spatial range covered by the hologram plane.
- **z-base** and **z-span**: define the physical depth range used during propagation.
- **splat-sigma**: controls the smoothing or spreading of projected points on the hologram plane.

These parameters allow the framework to test different storage-quality trade-offs. For example, a smaller **holo-size** may improve compression because the output images become smaller, but it may also lose fine spatial details. A larger number of **depth-bins** may preserve depth more accurately, but it also increases the complexity of the encoding and decoding process. Similarly, using more views may improve three-dimensional reconstruction, but it increases the number of stored amplitude and phase images.

6.3 View Configuration

A major capability of the framework is the ability to encode the object from different view configurations. The simplest configuration is a single view, where the object is observed from one direction. This produces the smallest encoded package, but it also loses the most depth information. For simple surfaces, a single view may be enough, but for closed three-dimensional shapes such as cubes, one view cannot fully describe all sides of the object.

The **cube6** setting uses six orthogonal views: front, back, right, left, top, and bottom. This is the most important view mode for the early experiments because it provides multi-directional evidence about the object. In holographic decoding, this helps separate true object structure from artifacts that may appear in only one projection. The framework also supports circular view modes, such as **circle8** or **circle12**, where views are distributed around the object. These settings can be useful for testing whether reconstruction improves when the object is observed from many angles.

This view system is especially relevant for the later LLM-compression stage. If weight matrices are converted into three-dimensional structures, different views may capture different aspects of the matrix geometry. Therefore, the framework allows experiments not only with compression level, but also with the amount of geometric information given to the decoder.

6.4 Decoding Module

The decoding module reconstructs a point cloud from the stored amplitude and phase images. First, the amplitude and phase images are loaded and combined to recover the complex hologram field. Then, for each view and each depth bin, the framework performs backward propagation. This produces an estimated intensity map for that depth. Instead of directly converting all bright pixels into points, the framework selects only the strongest candidate pixels according to a threshold and top- k rule.

The most important feature of the decoder is the confidence-volume approach. The decoder creates a shared three-dimensional voxel grid. Candidate points from each back-propagated view are mapped into this grid. Each view contributes confidence values to the voxels it supports. The result is a three-dimensional confidence volume:

$$C(x, y, z),$$

where higher values indicate stronger evidence that a voxel belongs to the original object.

This design improves reconstruction because it does not trust a single view independently. A voxel is considered more reliable when it is supported by multiple views. The decoder can require a minimum number of supporting views, remove weak-confidence voxels, and apply connected-component filtering to remove isolated noise. Therefore, the reconstructed point cloud is extracted only from the surviving high-confidence regions.

The decoding parameters include:

- **volume-resolution**: defines the size of the three-dimensional confidence grid.
- **min-view-support**: requires a voxel to be supported by at least a certain number of views.
- **support-radius**: expands per-view support to nearby voxels.
- **threshold-percentile**: keeps only strong pixels in each back-propagated depth slice.

- **volume-confidence-percentile**: removes weak voxels after multi-view fusion.
- **decode-oversample**: controls the number of candidate points considered before volume fusion.
- **top-k-per-slice**: manually controls how many strong pixels are selected per depth slice.
- **decoder-smooth-sigma**: optionally smooths the reconstructed intensity maps.
- **clip-extent**: defines the spatial range of the confidence volume.
- **confidence-gamma**: controls how strongly confidence values influence point selection.
- **jitter-fraction**: optionally adds small random displacement within selected voxels.
- **force-target-points**: forces the reconstructed point cloud to match the original point count.
- **max-output-points**: caps the number of reconstructed points.
- **min-output-points**: sets a minimum number of points for visualization.
- **connected-filter**: keeps the largest connected confidence components.

These settings allow detailed exploration of the reconstruction process. For example, increasing **min-view-support** may reduce noise because reconstructed points must be confirmed by multiple views. However, if this value is too high, some real points may be removed. Increasing **volume-resolution** gives a finer reconstruction grid, but also increases computational cost. A high **threshold-percentile** keeps only the brightest evidence, which may improve precision but reduce recall. Connected filtering can remove isolated artifacts, but it may also remove small valid components if the object has disconnected structures.

6.5 Quality-First Reconstruction

The framework uses a quality-first reconstruction strategy. By default, it does not force the reconstructed point cloud to contain exactly the same number of points as the original object. This is important because forcing the same point count can create artificial points in locations where the decoder has weak evidence. Such points may improve visual density but damage strict geometric metrics.

Instead, the decoder outputs points only from confident surviving voxels. If the confidence volume contains fewer reliable voxels, the reconstructed object will contain fewer points. This makes the evaluation more honest because the reconstruction reflects what the hologram package actually preserved. Exact point-count reconstruction is still supported through the **force-target-points** setting, but it is treated as an optional mode rather than the default.

This distinction is important for the capstone project. In model compression, artificially reconstructing extra parameters without reliable information may lead to numerical noise. Therefore, the quality-first setting is more aligned with the long-term goal of preserving meaningful structure rather than simply matching the original file size or point count.

6.6 Automated Experiment Mode

The framework also includes an automated experiment mode. In this mode, the user provides an input VTP file and an experiment directory. The framework then performs encoding, decoding, and reporting automatically. The output directory contains the encoded hologram package, the reconstructed VTP file, and strict metric reports.

The automated experiment structure is:

```
input.vtp → encoded.hologram/ → reconstructed.vtp → report/strict_metrics.
```

This mode is especially useful for parameter exploration. By changing parameters such as **holo-size**, **depth-bins**, **volume-resolution**, or **min-view-support**, the same object can be tested under many different compression and reconstruction conditions. Each experiment produces comparable output files and metrics, allowing the project to identify which settings produce the best balance between compression ratio and reconstruction accuracy.

6.7 Evaluation and Reporting Capability

The framework includes a strict reporting module that compares the original and reconstructed point clouds. The report includes point counts, file sizes, compression ratio, nearest-neighbor reconstruction errors, symmetric Hausdorff distance, precision, recall, F1 score, voxel Intersection over Union, and optional cube-surface metrics.

The nearest-neighbor metrics are computed in two directions. The original-to-reconstructed distance measures how well the reconstructed object covers the original object. The reconstructed-to-original distance measures how much of the reconstructed object lies close to the original. This distinction is important because a reconstruction may cover the original well while also producing many extra noisy points, or it may be precise but incomplete.

The framework also computes precision, recall, and F1 score under several distance thresholds. These metrics are useful because they convert geometric distance into an interpretable quality measure. Voxel IoU measures the overlap between occupied spatial cells in the original and reconstructed objects. For cube-like objects, additional metrics measure how many reconstructed points fall outside the original bounding box and how close the reconstructed points are to the nearest cube face.

6.8 Parameter Exploration Strategy

The large number of available parameters makes the framework suitable for systematic experimentation. The main experimental strategy is to vary one group of parameters at a time while observing both compression and reconstruction quality.

The first group is the storage-related encoding parameters. These include **holo-size**, **depth-bins**, and **views**. Reducing **holo-size** usually improves compression, because the amplitude and phase images become smaller. However, it may also reduce the spatial resolution of the encoded object. Reducing the number of views also decreases storage cost, but it may weaken depth reconstruction. Increasing **depth-bins** may improve depth representation, but it can make decoding more computationally expensive.

The second group is the reconstruction-filtering parameters. These include **min-view-support**, **threshold-percentile**, **volume-confidence-percentile**, and **connected-filter**. These parameters control how strict the decoder is. A stricter decoder removes more noise, but it may also remove valid parts of the object. A more permissive decoder keeps more points, but it may introduce artifacts.

The third group is the output-control parameters. These include **force-target-points**, **max-output-points**, **min-output-points**, and **jitter-fraction**. These parameters affect the final reconstructed point cloud rather than the hologram itself. They are useful for testing whether visual quality, point-count matching, or strict numerical accuracy should be prioritized.

Overall, the framework supports a controlled investigation of the compression–reconstruction trade-off. It allows the project to answer questions such as:

- How much can the hologram size be reduced before reconstruction quality collapses?
- How many views are necessary for stable three-dimensional reconstruction?
- Does phase storage improve reconstruction compared with amplitude-only storage?
- Does multi-view confidence filtering reduce noise compared with direct peak extraction?
- Which settings produce the best balance between compression ratio and geometric accuracy?

6.9 Connection to LLM Weight Compression

Although the current framework operates on VTP point clouds, it is designed as a preparation stage for LLM weight compression. The first goal is to test whether simple three-dimensional structures can be compressed and reconstructed using a holography-inspired representation. Once this is established, neural network weight matrices can be converted into similar spatial structures.

A weight matrix can be represented as a surface where the row index corresponds to one spatial axis, the column index corresponds to another spatial axis, and the weight value corresponds to height, depth, density, or intensity. Multiple matrices can be stacked or packed together to form a three-dimensional object. The same encoding pipeline can then be applied:

Weight matrices $\rightarrow$ 3D spatial representation $\rightarrow$ Amplitude/phase hologram package $\rightarrow$ Reconstructed spatial representation $\rightarrow$

Therefore, the framework is not only a tool for reconstructing cubes or synthetic shapes. It is a testing environment for studying whether holographic principles can preserve structured numerical data under compression. The experiments with simple objects provide the necessary foundation before moving to more complex and sensitive data, such as LLM parameters.

7 Spatial Representation of Weight Matrices

Before applying the holography-inspired encoding process, the numerical data must be converted into a spatial format. This step is necessary because the implemented framework operates on VTP point-cloud objects rather than directly on raw matrix files. Therefore, one of the important methodological questions in this project is how a collection of weight matrices should be represented as a three-dimensional object.

This representation stage is important because the quality of reconstruction depends not only on the holographic encoder and decoder, but also on how the original numerical data is transformed into geometry. If the matrix-to-VTP conversion loses important numerical structure, then even a strong holographic decoder cannot fully recover the original information. For this reason, several representation strategies were considered during the project.

7.1 Normalization as a Common Preprocessing Step

Normalization is applied before converting matrices into spatial objects. Neural network weight matrices may contain very small, very large, positive, and negative values. If these values are used directly as spatial coordinates, the resulting object may become distorted or difficult to encode. Therefore, all matrix values are normalized into the interval $[-1, 1]$.

The normalization formula is:

$$W'_{ij} = 2 \cdot \frac{W_{ij} - W_{\min}}{W_{\max} - W_{\min}} - 1,$$

where W_{ij} is the original matrix value, $W_{\min}$ is the minimum value in the matrix or matrix collection, and $W_{\max}$ is the maximum value. This transformation maps the smallest value to -1 , the largest value to 1 , and all intermediate values into the same controlled range.

This normalization is useful for three-dimensional representation because positive and negative values can be represented as positions above or below a central reference level. It also prevents a small number of large values from dominating the geometry. In this project, normalization to $[-1, 1]$ is treated as the common preprocessing step for all matrix-to-VTP representation strategies.

7.2 Single Large Square Matrix Representation

One representation strategy is to combine all selected weight matrices into one large square matrix before converting them into a VTP object. In this approach, the original matrices are placed together inside a larger two-dimensional canvas. If the model contains matrices

$$W = \{W_1, W_2, \dots, W_n\},$$

then these matrices are arranged into one combined matrix:

$$W_{\text{big}} = \begin{bmatrix} W_1 & W_2 & \dots \\ W_3 & W_4 & \dots \\ \vdots & \vdots & \ddots \end{bmatrix}.$$

When necessary, padding is added so that the final representation becomes square. This produces a single large matrix that contains information from many model components. After this combined square matrix is created and normalized, it is converted into a VTP point cloud using the mapping:

$$W_{\text{big}}(i, j) \rightarrow (x = i, y = j, z = W'_{\text{big}}(i, j)).$$

Here, the row index becomes the x -coordinate, the column index becomes the y -coordinate, and the normalized matrix value becomes the z -coordinate. The resulting VTP file is a surface-like three-dimensional object whose height represents the numerical values of the combined matrix.

The advantage of this method is that all selected matrices can be encoded as one object. This makes the input more compact and easier to process through the holographic pipeline, since the encoder receives one large spatial structure instead of many separate files. It also makes the data more similar to an image-like surface, which is suitable for hologram generation.

However, this approach also has limitations. Since different weight matrices may have different shapes, arranging them into one square layout may require padding or artificial empty regions. These regions do not represent real model parameters, but they still become part of the spatial object. As a result, the holographic encoder may spend part of its capacity encoding empty or artificial structure. Therefore, the layout of the combined square matrix is an important design choice.

7.3 Stacked Matrix Representation

A more model-aware representation is to stack multiple matrices as layers in three-dimensional space. In this strategy, each matrix becomes one surface, and the matrices are placed at different depth levels. If the model contains matrices

$$W = \{W_1, W_2, \dots, W_n\},$$

then each matrix can be mapped as:

$$W_k(i, j) \rightarrow (x = i, y = j, z = k + \alpha W'_{k,ij}),$$

where k represents the matrix index, and α controls how strongly the normalized matrix values deform each layer.

This representation is closer to the structure of an LLM because the model itself is layered. Each Transformer block contains several matrices, and the full model can be interpreted as a sequence of matrix layers. By stacking matrices, the VTP object becomes a three-dimensional structure rather than one flat surface.

The main advantage of this approach is that it preserves the separation between different matrices. The decoder can reconstruct a layered object, and each reconstructed layer can later be converted back into an approximate matrix. However, this method also introduces challenges. If layers are placed too close together, they may interfere during encoding and decoding. If they are placed too far apart, the object may become too large for efficient holographic representation.

7.4 Hemisphere-Based Representation

Another explored strategy is to map matrix values onto a hemisphere-like surface. In this approach, the matrix is not represented as a flat plane. Instead, its indices are mapped to angular coordinates, and the normalized values influence the radius or height of points on a curved surface.

A simplified version of this mapping can be described as:

$$(i, j, W'_{ij}) \rightarrow (r, \theta, \phi),$$

where i and j determine angular position and W'_{ij} affects the radial distance. The resulting coordinates can then be converted into Cartesian form:

$$x = r \sin(\theta) \cos(\phi),$$

$$y = r \sin(\theta) \sin(\phi),$$

$$z = r \cos(\theta).$$

The motivation for this strategy is that holography is naturally related to wavefronts and spatial propagation. A curved representation may distribute information more evenly across a three-dimensional space than a flat surface. It may also reduce the dominance of one viewing direction, because the object is no longer simply a plane viewed from the front.

However, the hemisphere representation is more complex than the flat matrix surface. After reconstruction, the object must be mapped back from curved coordinates into the original matrix structure. This inverse mapping may introduce interpolation errors. Therefore, the hemisphere approach is useful as an experimental representation, but it requires careful evaluation.

7.5 Comparison of Representation Strategies

The different matrix-to-VTP strategies offer different advantages and disadvantages. The single large square matrix representation allows all selected matrices to be encoded together as one surface-like object, but it may introduce artificial padding and layout boundaries. The stacked representation better reflects the layered structure of neural networks, but it requires careful control of spacing between layers. The hemisphere representation is more physically inspired and may distribute information more naturally in three-dimensional space, but it is harder to invert back into exact matrix form.

Overall, the project treats matrix-to-VTP conversion as an essential part of the compression problem. The holographic encoder does not operate on abstract model parameters directly; it operates on spatial objects. Therefore, the way matrices are transformed into geometry strongly affects the final reconstruction. By testing several representation strategies, the project investigates which spatial form is most suitable for holography-inspired model compression.

8 Testing and Results

For testing purposes, the project did not immediately use a full BERT model. Instead, a selected subset of BERT weight matrices was used in order to make the experiments computationally manageable while still preserving the real structure of Transformer-based neural networks. The tested subset contained matrices from two selected BERT Transformer layers, denoted as layer 01 and layer 02. For each layer, six matrices were considered: the query matrix Q , key matrix K , value matrix V , attention output matrix, first feed-forward network matrix $FFN1$, and second feed-forward network matrix $FFN2$.

The attention-related matrices Q , K , V , and attention output each had dimensionality 768×768 . The feed-forward matrices had larger rectangular shapes: $FFN1$ had dimensionality 768×3072 , while $FFN2$ had dimensionality 3072×768 . Therefore, the tested BERT subset contained both square attention matrices and rectangular feed-forward matrices, making it suitable for evaluating different matrix-to-VTP representation strategies.

Matrix Type	Layer Index	Dimensionality
Q	01, 02	768×768
K	01, 02	768×768
V	01, 02	768×768
Attention Output	01, 02	768×768
$FFN1$	01, 02	768×3072
$FFN2$	01, 02	3072×768

Table 1: Selected BERT weight matrices used for testing the matrix-to-VTP and holographic encoding pipeline.

This subset was useful because it represents the main internal components of a BERT Transformer block. The Q , K , and V matrices are used in the self-attention mechanism, the attention output matrix projects the attention result back into the hidden representation space, and the two feed-forward matrices expand and compress the hidden representation through the intermediate dimension. Since BERT-base uses a hidden size of 768 and an intermediate feed-forward size of 3072, these matrices provide a realistic test case for the proposed compression framework.

8.1 Preliminary Reconstruction on Simple Three-Dimensional Objects

Before applying the holography-inspired framework to neural network weight matrices, an earlier version of the pipeline was tested on simple three-dimensional VTP objects. The purpose of this preliminary experiment was to check whether a hologram-like encoding could preserve recognizable spatial information from a scene containing multiple basic geometric components.

In this older pipeline, the input was not yet an LLM weight matrix. Instead, the input was a VTP point cloud containing several simple objects in one shared scene. The codec first normalized the point coordinates. Then, the normalized x and y coordinates were rasterized into a two-dimensional complex object field, while the normalized z -coordinate was encoded as phase. Therefore, the mapping used in this version can be described as a phase-depth surface representation:

$$(x, y, z) \rightarrow F(x, y) = A(x, y)e^{i\phi(z)}.$$

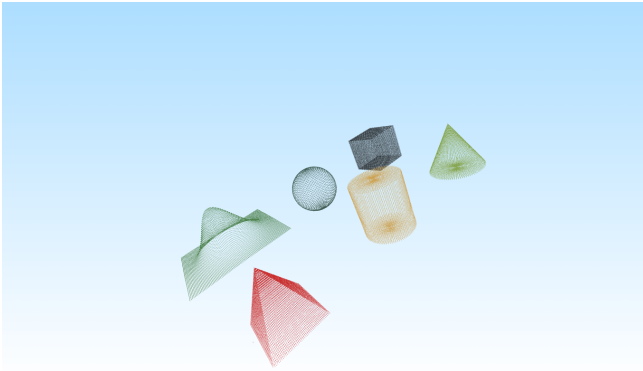
In this representation, the amplitude $A(x, y)$ mainly represents point density or occupancy, while the phase term $\phi(z)$ stores depth information. After constructing this complex object field, a two-dimensional Fourier transform was applied:

$$H = \mathcal{F}\{F(x, y)\},$$

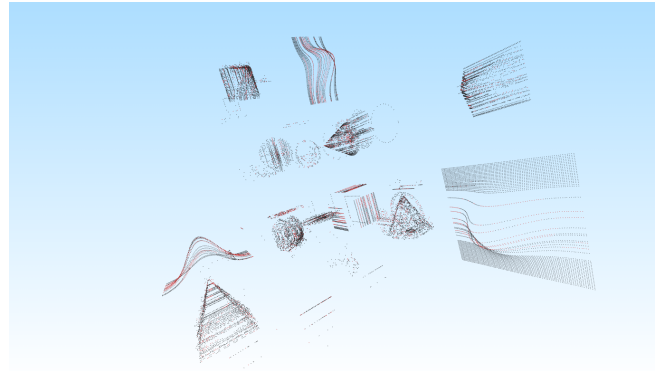
where H is the hologram-like encoded field. During decoding, the inverse Fourier transform was applied to recover the object field. Then, amplitude thresholding was used to select reconstructed points, and the recovered phase was used to estimate the reconstructed depth.

This mapping is closest to a height-field or surface-style representation. It is suitable when each (x, y) position has one main depth value. However, it is less suitable for complex three-dimensional objects where multiple points may share the same projected (x, y) location but have different z -values. In such cases, the phase values may interfere or average together, which can lead to incomplete reconstruction.

Figure 1 shows the result of this preliminary test. The original input file, shown in Figure 1a, contains several complete geometric objects in one scene. The reconstructed output, shown in Figure 1b, does not perfectly reproduce the original scene. Instead, the result appears as decomposed and separated components of the original objects. Some surfaces, outlines, and directional structures remain visible, but the reconstruction is fragmented and noisy.



(a) Original VTP scene containing multiple simple objects.



(b) Reconstructed output from the older phase-depth FFT pipeline.

Figure 1: Preliminary reconstruction of a multi-object VTP scene using the older hologram-like phase-depth FFT pipeline.

The result is important because it demonstrates both the potential and the limitation of the first approach. On one hand, the reconstruction is not random. The decoded output preserves traces of the original geometry and separates the scene into recognizable component-like structures. This suggests that the hologram-like encoding retained some information about the original VTP object.

On the other hand, the reconstruction is not geometrically accurate enough. The original objects are not recovered as clean, complete, and stable three-dimensional shapes. Instead, the output contains partial surfaces, scattered points, and separated fragments. This shows that the phase-depth FFT method can preserve some structural information, but it struggles with complex scenes containing overlapping depths.

This preliminary result motivated the development of the later confidence-volume pipeline. The newer approach uses multi-view encoding, separate amplitude and phase storage, depth-bin propagation, and confidence-based three-dimensional reconstruction. Therefore, this first experiment serves as a useful baseline: it shows that hologram-like encoding can preserve partial geometric structure, while also showing why stronger depth handling and multi-view reconstruction are necessary.

8.2 Full Matrix Roundtrip Reconstruction Experiment

After testing the earlier pipeline on simple three-dimensional objects, the next experiment moved closer to the real goal of the capstone by applying the framework to actual BERT weight matrices. This experiment used three related scripts. The first script performed the full matrix-aware roundtrip pipeline, converting a folder of matrix files into a grouped VTP point cloud, encoding it into hologram image files, decoding it back into CSV matrices, and reconstructing a grouped VTP file. The second script was designed as a split-and-merge utility for large matrices,

especially for feed-forward matrices that are larger than one standard attention matrix. The third script compared the original CSV matrices with the decoded CSV matrices and generated the final similarity report.

The matrix-aware pipeline represents each matrix as a spatial block inside one grouped VTP object. For every matrix element, the column index determines the x -coordinate, the row index determines the y -coordinate, and the matrix value determines the z -coordinate:

$$x = \text{col_index} \cdot x_{\text{spacing}} + x_{\text{offset}},$$

$$y = \text{row_index} \cdot y_{\text{spacing}} + y_{\text{offset}},$$

$$z = W_{ij}.$$

The grouped VTP also stores matrix-specific metadata such as the original value, layer index, row index, and column index. This is important because the decoder must not only reconstruct a point cloud visually, but also recover the original matrix structure and save it back as CSV files.

In the encoding stage, the pipeline does not treat the group of matrices as one ordinary image. Instead, it uses a low-frequency FFT tiling strategy. Each matrix is first normalized internally, transformed using a two-dimensional Fourier transform, and then placed into a tile inside a larger spectrum canvas. The final hologram-like field is obtained by applying an inverse Fourier transform to this spectrum canvas. The encoded output is stored through companion image files, including real and imaginary PNG files, together with metadata required for decoding.

During decoding, the process is reversed. The real and imaginary PNG files are loaded to recover the complex field. A Fourier transform is applied to recover the spectrum canvas. Then, the tile corresponding to each matrix is extracted, pasted back into a spectrum of the matrix’s original shape, and transformed back into the spatial domain. The reconstructed matrices are then saved as decoded CSV files, and a reconstructed grouped VTP is also generated.

The split-and-merge script was introduced because large rectangular matrices can be harder to reconstruct in this tiling setup. In particular, the feed-forward matrices $FFN1$ and $FFN2$ are much larger than the 768×768 attention matrices. The utility therefore allows large matrices to be divided into smaller blocks, such as 768×768 , before hologram encoding. After decoding, the blocks can be merged back into their original matrix shapes. For this BERT-based experiment, $FFN1$ of size 768×3072 can be divided into four 768×768 blocks, and $FFN2$ of size 3072×768 can also be divided into four 768×768 blocks.

For evaluation, the decoded matrices were compared with the original matrices using a separate comparison script. The comparison script computes Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), maximum absolute error, relative Frobenius error, and similarity percentage. The similarity score is defined as:

$$\text{relative error} = \frac{\|W - \hat{W}\|_F}{\|W\|_F},$$

$$\text{similarity} = 100 \cdot (1 - \text{relative error}),$$

where W is the original matrix and $\hat{W}$ is the decoded matrix. This means that the similarity score is based on numerical reconstruction error, not on visual similarity or model accuracy.

The experiment used twelve BERT matrices from two Transformer layers. These included the query, key, value, attention output, first feed-forward, and second feed-forward matrices for layers 01 and 02. The attention-related matrices had shape 768×768 , while the feed-forward matrices had shapes 768×3072 and 3072×768 .

Matrix Group	Shape	Average MAE	Average RMSE	Average Similarity
Attention matrices Q, K, V , Output	768×768	0.204384	0.255446	25.55%
Feed-forward $FFN1$	768×3072	0.204344	0.255457	25.56%
Feed-forward $FFN2$	3072×768	0.204649	0.255790	25.49%
Overall average	Mixed	0.204422	0.255505	25.54%

Table 2: Average reconstruction results for the full matrix roundtrip experiment.

The detailed layer-by-layer results are shown in Table 3. The similarity values are very close across all matrices, ranging from approximately 25.38% to 25.66%. This shows that the reconstruction behavior was stable across different matrix types, but the absolute reconstruction quality was still limited.

Original Matrix	Shape	MAE	RMSE	Similarity
<i>FFN1_01</i>	768×3072	0.204329	0.255413	25.57%
<i>FFN1_02</i>	768×3072	0.204360	0.255501	25.56%
<i>FFN2_01</i>	3072×768	0.204643	0.255785	25.48%
<i>FFN2_02</i>	3072×768	0.204655	0.255794	25.49%
<i>K_01</i>	768×768	0.204658	0.255758	25.53%
<i>K_02</i>	768×768	0.204737	0.255848	25.38%
<i>Q_01</i>	768×768	0.204679	0.255823	25.57%
<i>Q_02</i>	768×768	0.203974	0.254852	25.65%
<i>V_01</i>	768×768	0.203656	0.254556	25.66%
<i>V_02</i>	768×768	0.204610	0.255732	25.47%
Attention Output 01	768×768	0.204277	0.255359	25.58%
Attention Output 02	768×768	0.204483	0.255641	25.53%

Table 3: Layer-by-layer reconstruction results from the decoded CSV similarity report.

The result of this experiment is important because it demonstrates that the full reconstruction pipeline was technically successful: matrix files were converted into a grouped VTP representation, encoded into hologram-like files, decoded back into CSV matrices, and compared numerically against the originals. However, the numerical similarity remained low, at approximately 25.54% on average. This indicates that although the framework can perform a complete roundtrip reconstruction, the decoded matrices are still far from the original BERT matrices in strict numerical terms.

The main reason for this limitation is that the FFT tiling approach preserves only part of each matrix’s frequency information. When a matrix spectrum is cropped into a limited tile, high-frequency details are lost. This is especially problematic for neural network weights, where small numerical differences may matter even if the reconstructed matrix visually appears structurally similar. Therefore, this experiment showed that a complete matrix reconstruction workflow is possible, but stronger encoding and decoding methods are necessary before the approach can be considered effective for model compression.

Overall, this experiment represents a transition from simple geometric reconstruction to actual neural network parameter reconstruction. Compared with the simple-object test, this experiment is more directly connected to the capstone goal because it reconstructs BERT matrix data rather than synthetic VTP geometry. At the same time, the low similarity score motivates later improvements, such as better preservation of frequency components, larger hologram sizes, block-based reconstruction, phase-aware encoding, and confidence-based decoding.

8.3 Cube Reconstruction Using the Confidence-Volume Decoder

The third experiment used a newer version of the VTP holography pipeline, called the confidence-volume decoder. Unlike the earlier FFT-based phase-depth codec, this tool is designed specifically for more reliable three-dimensional reconstruction. Its purpose is to encode a VTP point cloud into a hologram-like package and then reconstruct the object using multi-view evidence.

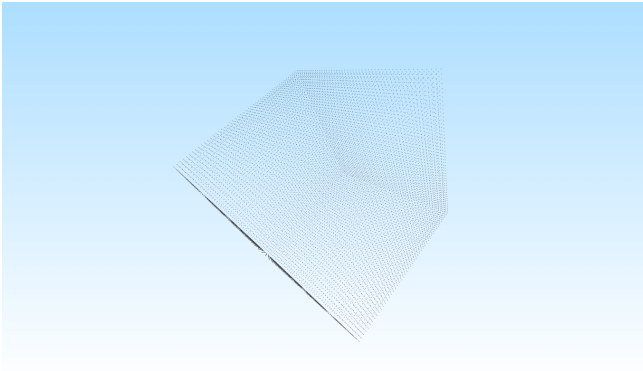
The pipeline follows the structure:

$$\text{initial VTP} \rightarrow \text{amplitude} + \text{phase} + \text{metadata} \rightarrow \text{confidence-volume decoding} \rightarrow \text{reconstructed VTP}.$$

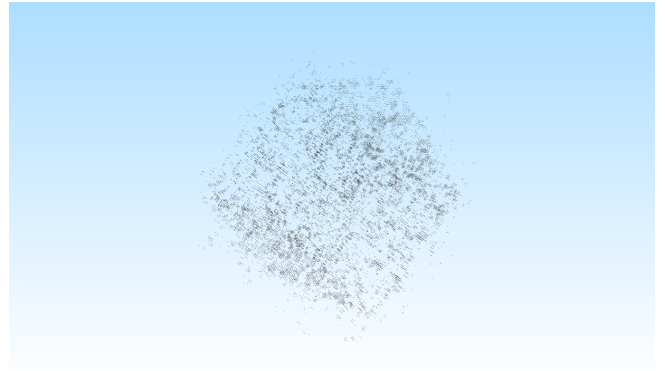
In the encoding stage, the input VTP point cloud is first normalized into a common coordinate system. Then the object is observed from multiple views. In the cube experiment, the multi-view setup is important because a cube cannot be fully represented from only one direction. For each view, the object is divided into depth bins. Each depth slice is projected onto a two-dimensional complex plane and propagated using an angular-spectrum-style wave propagation method. The result is a complex hologram field for each view. This field is stored using two images: an amplitude PNG and a phase PNG. A metadata file stores the normalization parameters, view rotations, depth-bin information, hologram size, and other reconstruction settings.

The main improvement of this tool is in the decoding stage. Instead of directly taking the brightest reconstructed pixels as final points, the decoder back-propagates each view through the depth bins and collects candidate evidence in a shared three-dimensional confidence volume. A voxel is treated as more reliable when it receives support from multiple views. The decoder can also remove weak evidence and isolated components. Therefore, the reconstruction is based on multi-view agreement rather than one projection alone.

This experiment was applied to a cube object, `cube_average.vtp`. Figure 2 compares the original cube and the reconstructed result. Visually, the reconstruction preserves the general cube-like structure. The reconstructed object is not a perfect copy of the original cube, but it shows that the newer confidence-volume decoder is much more suitable for closed three-dimensional objects than the older phase-depth FFT method.



(a) Original cube object.



(b) Reconstructed cube using the confidence-volume decoder.

Figure 2: Cube reconstruction using the newer Gabor-style confidence-volume decoder.

The numerical results are summarized in Table 4. The original file contained 13,256 points, while the reconstructed file contained 18,000 points. The encoded hologram package had a size of 384,468 bytes, compared with the original VTP size of 674,302 bytes. This gives a compression ratio of approximately $1.75\times$. Therefore, the method achieved actual compression, although the compression level was moderate rather than extreme.

Metric	Value
Original points	13,256
Reconstructed points	18,000
Original VTP size	674,302 bytes
Encoded hologram size	384,468 bytes
Compression ratio	$1.7539\times$
Original-to-reconstructed RMSE	0.056166
Reconstructed-to-original RMSE	0.119892
Symmetric Hausdorff 95	0.244163
Symmetric Hausdorff max	0.819432
F1 at one estimated spacing	44.87%
F1 at two estimated spacings	78.91%
Voxel IoU at resolution 128	1.45%
Outside original bounding box	37.81%

Table 4: Strict reconstruction metrics for the cube confidence-volume experiment.

The nearest-neighbor distances show that the reconstruction captured the cube structure partially but not perfectly. The original-to-reconstructed RMSE was 0.056166, while the estimated original point spacing was 0.042553. This means that many original cube points had a reconstructed point reasonably nearby, although not exactly at the same location. The reconstructed-to-original RMSE was higher, 0.119892, which suggests that the reconstructed point cloud contained extra or displaced points that did not lie as close to the original cube surface.

The precision, recall, and F1 results give a more balanced interpretation. At a strict threshold of one estimated point spacing, the F1 score was 44.87%. At two spacings, the F1 score increased to 78.91%. This suggests that the reconstructed cube is geometrically close at a broader tolerance, even though exact point-level reconstruction remains difficult. At larger thresholds, the F1 score becomes very high, reaching 95.94% and 99.63%, which means that the reconstructed object remains generally near the original cube region.

However, the voxel IoU values are low, especially at high resolution. The voxel IoU was only 1.45% at resolution 128 and 0.00% at resolution 256. This indicates that exact voxel-level overlap between the original and reconstructed point clouds is weak. In other words, the reconstructed object has a similar general shape, but its points do not occupy the exact same fine-grid cells as the original object. This is expected for a noisy holographic reconstruction, but it also shows that the method is not yet precise enough for exact geometric recovery.

The cube-surface metrics also show an important limitation. About 37.81% of reconstructed points were outside the original bounding box. This means that the decoder still produces a significant amount of off-surface or outside-region evidence. At the same time, the average distance to the nearest cube face was 0.075586, which suggests that many reconstructed points still remain close to the cube surface. Therefore, the reconstruction is not random, but it is also not clean enough to be considered a highly accurate cube recovery.

Overall, this experiment is a meaningful improvement over the older simple-object reconstruction. The confidence-volume decoder reconstructs a recognizable cube-like object and achieves moderate compression. The result shows that multi-view amplitude-and-phase encoding combined with confidence-volume decoding is a better direction for

three-dimensional reconstruction. At the same time, the strict metrics show that the method still has limitations: the reconstruction contains extra points, has low fine-scale voxel overlap, and places a noticeable fraction of points outside the original bounding box. Therefore, this experiment supports the feasibility of the approach, but also shows that further refinement is needed before applying the same idea to sensitive numerical data such as LLM weight matrices.

9 Future Work

The current project mainly focuses on designing and testing a holography-inspired encoding and decoding framework through algorithmic reconstruction. The experiments show that geometric and matrix-based information can be encoded into hologram-like representations and reconstructed to some extent. However, the reconstruction quality is still limited, especially when strict numerical accuracy is required. Therefore, the most important direction for future work is to replace or support the hand-designed decoder with deep learning models trained specifically for hologram reconstruction.

A promising future direction is to train image-to-image or image-to-volume neural networks, such as U-Net, to reconstruct the original object or matrix representation from the encoded hologram. U-Net is especially suitable for this task because it is designed to preserve both global structure and local details through an encoder-decoder architecture with skip connections. In the context of this project, the encoded amplitude and phase images can be used as input channels, while the target output can be the original matrix, reconstructed depth map, point-cloud occupancy grid, or a voxelized representation of the original VTP object.

The future learned reconstruction pipeline can be described as:

$$H_{\text{amp}}, H_{\text{phase}} \rightarrow \mathcal{N}_{\theta}(H) \rightarrow \hat{P},$$

where H_{amp} is the amplitude hologram, H_{phase} is the phase hologram, $\mathcal{N}_{\theta}$ is a trained neural network decoder, and $\hat{P}$ is the reconstructed spatial representation. For matrix reconstruction, the output could instead be written as:

$$H_{\text{amp}}, H_{\text{phase}} \rightarrow \mathcal{N}_{\theta}(H) \rightarrow \hat{W},$$

where $\hat{W}$ is the reconstructed weight matrix or group of matrices.

This approach would change the decoder from a manually designed reconstruction rule into a learned inverse mapping. The current decoder depends on fixed operations such as back-propagation, thresholding, confidence-volume voting, and connected-component filtering. These steps are useful, but they may not fully capture the complex relationship between the encoded hologram and the original object. A neural decoder could learn this relationship directly from many examples. For example, the training dataset could contain pairs of original objects and their encoded holograms:

$$(P_1, H_1), (P_2, H_2), \dots, (P_n, H_n),$$

or, in the matrix case:

$$(W_1, H_1), (W_2, H_2), \dots, (W_n, H_n).$$

The model would then learn to minimize the difference between the original data and the reconstructed output. Possible loss functions include Mean Squared Error (MSE), Mean Absolute Error (MAE), cosine similarity loss, structural similarity loss, Chamfer distance for point clouds, and voxel IoU-based losses for three-dimensional reconstructions. For LLM weight matrices, the loss function may also include matrix-level similarity measures such as relative Frobenius error:

$$\mathcal{L}_F = \frac{\|W - \hat{W}\|_F}{\|W\|_F}.$$

In addition to numerical similarity, a future version of the project should evaluate whether reconstructed matrices preserve model behavior. This means that after reconstructing the weights, they should be inserted back into a model and tested on downstream tasks. This would answer a more important question: not only whether the reconstructed weights are numerically close, but whether they preserve the functional behavior of the original neural network.

Several deep learning architectures could be explored. A standard two-dimensional U-Net could be used when the hologram is treated as a multi-channel image, where amplitude and phase are separate channels. A three-dimensional U-Net could be used if the target is a voxelized object or a multi-layer matrix volume. Autoencoders could be used

to learn compact latent representations of holograms. Hybrid physics-informed networks could also be developed, where the model combines wave-propagation operations with learnable neural layers. This would allow the future system to benefit from both holographic principles and data-driven reconstruction.

Another important direction is training on synthetic data before moving to real model weights. A large dataset of simple geometric objects, surfaces, cubes, spheres, bell-shaped functions, and artificial matrix patterns could be generated automatically. The model could first learn general reconstruction behavior from these controlled examples. After that, it could be fine-tuned on actual neural network weight matrices, such as BERT attention and feed-forward layers. This staged training process would be similar to the current experimental structure of the project, where simple 3D objects are tested before moving to real LLM matrices.

The main limitation of this future direction is computational resources. Training a neural reconstruction model requires a large dataset of encoded holograms and corresponding ground-truth objects or matrices. It also requires GPU resources, memory, and time for repeated training and evaluation. For small experiments, a U-Net could be trained on lower-resolution holograms and smaller matrix blocks. However, for realistic LLM-scale compression, the training process would require much larger datasets and stronger hardware. Therefore, the use of deep learning is not only a methodological question, but also a matter of available resources.

Future work should also explore block-based learning. Large matrices such as 768×3072 or 3072×768 can be split into smaller blocks, reconstructed independently, and then merged back together. This would reduce memory requirements and make neural decoding more practical. A learned decoder could be trained on fixed-size matrix blocks, such as 768×768 or smaller patches, instead of trying to reconstruct the entire model at once.

Another possible improvement is to train the model to predict correction terms rather than reconstructing the full matrix directly. In this approach, the current algorithmic decoder would first produce an approximate reconstruction $\hat{W}_0$. Then, a neural network would learn the residual error:

$$R = W - \hat{W}_0.$$

The final reconstruction would be:

$$\hat{W} = \hat{W}_0 + \mathcal{N}_\theta(H).$$

This residual-learning strategy may be easier than full reconstruction because the model only needs to correct the systematic errors of the holographic decoder.

Overall, the future direction of this project is to move from purely algorithmic decoding toward learned hologram reconstruction. The current framework provides the foundation: it can generate encoded hologram packages, reconstructed outputs, and strict metrics. A future deep learning system could use this framework to generate training data and evaluate reconstruction quality. If sufficient computational resources are available, models such as U-Net could significantly improve reconstruction accuracy and make holography-inspired LLM compression more practical.

10 References

References

- [1] Gabor, D. (1948). A new microscopic principle. *Nature*, 161, 777–778. <https://doi.org/10.1038/161777a0>
- [2] Goodman, J. W. (2005). *Introduction to Fourier Optics* (3rd ed.). Roberts and Company Publishers.
- [3] Joint Photographic Experts Group. (2024). *White Paper on JPEG Pleno Holography*. JPEG. https://ds.jpeg.org/whitepapers/wg1n101066-106-COM-White_Paper_on_JPEG_Plano_Holography.pdf
- [4] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*. <https://arxiv.org/abs/1706.03762>
- [5] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT 2019* (pp. 4171–4186). Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>
- [6] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015* (pp. 234–241). Springer. https://doi.org/10.1007/978-3-319-24574-4_28

- [7] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*. <https://arxiv.org/abs/1910.01108>
- [8] Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*. <https://arxiv.org/abs/2210.17323>
- [9] Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2023). SmoothQuant: Accurate and efficient post-training quantization for large language models. In *Proceedings of the 40th International Conference on Machine Learning*. <https://arxiv.org/abs/2211.10438>
- [10] Frantar, E., & Alistarh, D. (2023). SparseGPT: Massive language models can be accurately pruned in one-shot. In *Proceedings of the 40th International Conference on Machine Learning*. <https://arxiv.org/abs/2301.00774>
- [11] Van Rossum, G., & Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace.
- [12] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., Gérard-Marchant, P., Sheppard, K., Reddy, T., Weckesser, W., Abbasi, H., Gohlke, C., & Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, *585*, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- [13] Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., van Mulbregt, P., & SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, *17*, 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
- [14] McKinney, W. (2010). Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference* (pp. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- [15] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- [16] Schroeder, W., Martin, K., & Lorensen, B. (2006). *The Visualization Toolkit: An Object-Oriented Approach to 3D Graphics* (4th ed.). Kitware.
- [17] Pillow Contributors. (n.d.). *Pillow: The Friendly PIL Fork*. Pillow documentation. <https://pillow.readthedocs.io/>